Python Engineering - Classes & Objects

Entity, class, object, attributes, methods, modules and self

This guide explains the basic object-oriented programming vocabulary in simple terms. The goal is not to memorize definitions, but to understand how the pieces fit together.

1. Entity: the thing we want to represent

An **entity** is a meaningful thing in the problem that we want to model: an aircraft, satellite, sensor, IMU, or in EX21 a vertical-motion state.

Conceptually, we want something that has height h, velocity v, time t, and gravity g.

2. Class: the general definition

A **class** describes what objects of that type will store and what operations they can perform.

```
class VerticalState:
    ...
```

Think of the class as a blueprint rather than a concrete state.

3. Object: one concrete realization

An **object** or **instance** is a concrete realization created from a class.

```
earth_state = VerticalState(h=100.0, v=20.0, g=9.81)
moon_state  = VerticalState(h=100.0, v=20.0, g=1.62)
```

Both objects share the same class definition but store independent state.

4. Attributes: data stored by the object

```
earth_state.h
earth_state.v
earth_state.t
earth_state.g
```

Attributes answer: **what does this object know or contain?**

5. Methods: what the object can do

A **method** is a function defined inside a class and intended to operate on an object of that class.

```
def specific_energy(self):
    return self.g * self.h + 0.5 * self.v**2

def euler_step(self, dt):
    self.h = self.h + self.v * dt
    self.v = self.v - self.g * dt
    self.t = self.t + dt
```

Attributes are what the object stores; methods are what the object can do.

6. What does self mean?

self means this particular object. It tells the method which instance it is currently working with.

```
earth_state.specific_energy()
# self -> earth_state

moon_state.specific_energy()
# self -> moon_state
```

self.h means the current object's height; self.g means its stored gravity.

7. h versus self.h

```
def __init__(self, h, v, t=0.0, g=9.81):
    self.h = h
```

h is a temporary incoming parameter. self.h is the attribute stored on the object.

Right-hand side: incoming value. Left-hand side: where the object stores it.

8. __init__: preparing the object

__init__ initializes object state when the object is created.

```
earth_state = VerticalState(h=100.0, v=20.0)

# conceptually:
# create object -> call __init__ -> store attributes
```

9. Module: the .py file that organizes code

A **module** is simply a .py file containing reusable Python code. It can contain functions, classes, constants, and other definitions.

```
vertical_state.py
    contains -> class VerticalState

main.py
    imports -> VerticalState
```

Module, class, and object are different levels.

10. Complete mental hierarchy

```
ENTITY -> vertical-motion state
MODULE -> vertical_state.py
CLASS  -> VerticalState
OBJECTS -> earth_state, moon_state
ATTRIBUTES -> h, v, t, g
METHODS -> specific_energy(), euler_step(), ...
self -> 'this particular object'
```

11. Aircraft example

```
class Aircraft:
    # attributes: mass, position, velocity, fuel, attitude
    # methods: calculate_lift(), calculate_drag(), burn_fuel()

aircraft_1 = Aircraft(...)
aircraft_2 = Aircraft(...)
```

Calling aircraft_1.burn_fuel() changes aircraft_1, not aircraft_2.

Final summary

Concept	Simple idea
Entity	The thing in the problem we want to represent
Module	.py file organizing reusable code
Class	General definition / blueprint
Object	Concrete instance of a class
Attribute	Data stored on the object
Method	Class function operating on the object
self	Reference to the current concrete object
__init__	Initializes object attributes